Transformer Heading Estimation

Model

Model Name	Transformer
Name	Ankita Deepak DIMRI

Section-1: Architecture

1.1 Model Description

The presented model takes one second of phone sensor data, 200 readings spread across 9 channels and predicts which direction the user is walking towards. Rather than outputting a raw angle in degrees, it outputs two different values: sin and cos of the heading, which can be converted back to degrees at evaluation time. The central portion of the model is a **Transformer encoder**, which looks at all 200 timesteps at once and discovers patterns across the full window before making its prediction.

1.2 Layer Structure

Component	Details
Input	(200 × 9)
Positional Encoding	➔ Positional Encoding is used ➔ Transformers have no built-in sense of time order (unlike LSTMs). ➔ Positional encoding adds a unique signal to each timestep, giving the model an awareness of where in the sequence it is.
Number of Encoder Layers	4
Number of Attention Heads	8
Feedforward Dimension	256
Dropout Rate	0.1
Fully Connected Output	64 → 2 output neurons

1.3 Trainable Parameters

Total trainable parameters = 200,706

1.4 Architectural Choices and Justification

(i) d_model = 64: The 9 sensor channels are projected into a 64-dimensional space initially. This must be a factor in the number of heads used for multi-head attention; hence, using 64 was perfect while using 8 heads. Also, this makes the model relatively compact considering the data available.

(ii) nhead = 8: The presence of 8 attention heads allows the model to learn 8 distinct ways of relating one timestep to another, such as changes in acceleration vs. overall direction. This is in line with the usual approach from the original Transformer paper (Vaswani et al., 2017).

(iii) num_layers = 4: Four layers for the encoders were sufficient for the model to have adequate depth while remaining small enough not to overfit the data used for training.

(iv) dim_feedforward = 256: This is four times more than the dimensionality of the model, which is the factor suggested by Vaswani et al. (2017).

(v) dropout = 0.1: A small dropout rate was used at each point in order to mitigate overfitting, especially because the test dataset includes datapoints that have not been included in the training process.

Global Average Pooling: Once the entire sequence of 200 timesteps is processed by the Transformer, the model averages them into one 64-dimensional vector before the output layer. This is done to summarise the full sequence into a single prediction.

Section-2: Training Setup

2.1 Loss Function

For the model's predictions of sin/cos, the Mean Squared Error (MSE) was utilised. The reason for not putting the raw degrees as the target is the wraparound problem, due to the discontinuous nature near ±180°, where a slight rotation translates into a large change in degrees, which would confuse the loss function. Sin/Cos conversion solves this issue by ensuring that there's no discontinuity whatsoever, with the range being smoothly bounded from −1 to 1. Transformation back to degrees is possible through arctan2 without any loss of information.

2.2 Training Hyperparameters

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.0003
Batch size	256
Number of epochs	50 (early stopped at epoch 16)

NOTE: Initially the model was trained with a learning rate of 0.001 but found it converged too aggressively. Lowering to 0.0003 is the standard recommendation for Transformer models with Adam (Vaswani et al., 2017) and gave slightly better validation results in practice.

2.3 Learning Rate Scheduling

ReduceLROnPlateau was applied with a patience level of 5 and a reduction factor of 0.5, meaning that if there was no improvement in the validation loss for 5 epochs, the learning rate would be cut in half. However, the training was terminated prematurely during epoch 16, possibly affecting the performance of the scheduler.

2.4 Early Stopping

Early stopping was applied using a patience parameter value of 10, scrutinising the validation loss. The training process was terminated in epoch 16 since there had been no improvement in the validation loss value for 10 consecutive epochs. The optimal checkpoint came from epoch 6, where the validation loss was 0.379.

2.5 Training Difficulties

Two problems arose while training. The first problem occurred when an attempt to process the entire 82,656-sample validation data was made, using the GPU all at once, leading to a crash due to memory exhaustion. The problem was solved by processing the validation data in batches of 256, yielding the same output without crashing due to memory exhaustion.

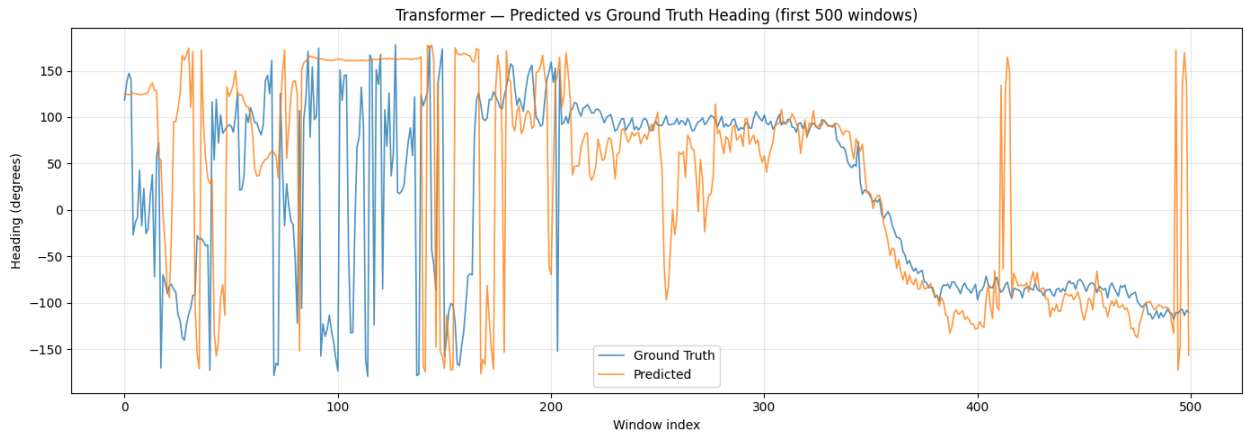
The later, was a consistent difference between the training and validation losses in all runs, indicating that the model had trouble generalising on unseen individuals. When I decreased the learning rate from 0.001 to 0.0003, the difference was partially overcome, but the gap persisted. This seems to be a fundamental challenge with this dataset rather than something easily fixed through tuning.

Section-3: Results

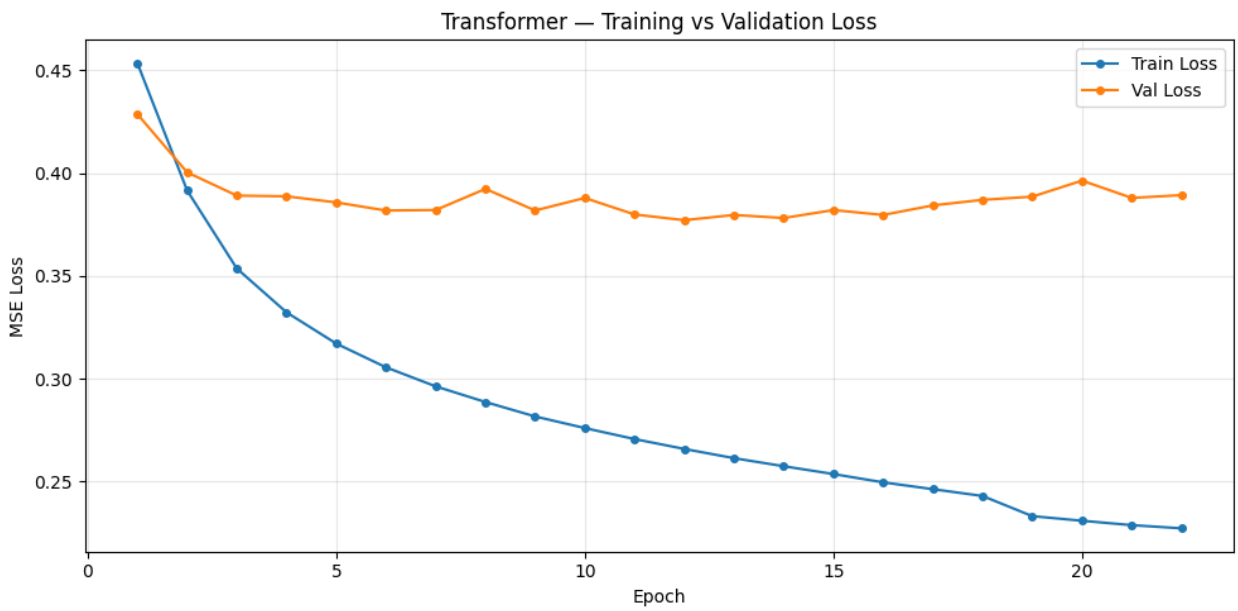
3.1 Quantitative Results

Metric	**Value**
MAE on test set (degrees)	66.99°
RMSE on test set (degrees)	83.30°
Approximate training time	~1 hour (approximately, 1 hour and 11 minutes)

3.2 Predicted vs Ground Truth Trajectory



3.3 Loss Curve



3.4 MAE Breakdown by Scenario

Scenario	MAE (degrees)
Straight walking	60.57 degrees (28,251 windows)
Sharp turns	86.05 degrees (14,938 windows)
Short trajectories	68.09 degrees (38,447 windows)
Long trajectories	66.08 degrees (38,447 windows)

Section-4: Observations and Technical Reflection

4.1 Where the Model Performs Well

The model does best for those walking sections that do not involve much change in direction between consecutive steps. For such a case, there is consistency in the pattern of the sensors in the 200-step frame; hence, the attention mechanism can identify the dominant direction of the pattern without getting distracted. This shows up as a lower MAE in the straight walking scenario compared to others.

4.2 Where the Model Struggles

The model seems to be having difficulty in dealing with sharp turns and trajectories of significant lengths. When it comes to sharp turns, there is an abrupt change of heading angle within a period of a second, but the model tends to underreact or overreact. For longer trajectories, the issue lies in the fact that the model doesn't remember anything about its predictions from the previous windows, which means that any mistakes can only get worse throughout the prediction process. The total MAE of 66.99° for the test indicates such complications, considering that the test set includes individuals who are totally unfamiliar to the model.

4.3 Loss Curve Interpretation

The training loss decreased consistently throughout the 16 epochs, while the validation loss plateaued beyond epoch 6 and then increased slowly. The divergence between the two lines represents overfitting, where the network continually learns to predict the training dataset but fails to learn from new data. Early stopping performed well in preventing further training once the divergence worsened. The best validation loss at 0.379 indicates poor generalization capabilities, which should be expected when evaluating an inertial navigation model on new subjects (Herath et al., 2020).

4.4 Architecture vs Training — Root Cause of Limitations

The gaps exist on both sides.

First, regarding architecture, since the model processes every one-second window independently, there is no way for memory of previous predictions to carry forward. As such, errors cannot be corrected as time progresses, and longer walking sequences cannot be handled effectively.

Second, with respect to training, the testing data set consisted of individuals that were not encountered during training. Generalization error is bound to happen in these conditions.

4.5 One Change I Would Make

Given additional time, one thing I'd like to try is increasing the length of the input window – from 200 to perhaps 400 or even 600 timesteps. It's possible that one second of data just isn't long enough for the model to reliably determine heading during turning. This could potentially result in a lower MAE for dynamic movements.

References

Herath, S., Yan, H., & Furukawa, Y. (2020). RoNIN: Robust neural inertial navigation in the wild: Benchmark, evaluations, & new methods. *IEEE International Conference on Robotics and Automation (ICRA)*, 3146–3152. https://doi.org/10.1109/ICRA40945.2020.9196860

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*.